

Grounding LLM-Enhanced Web Agents with User-Interaction Traces

Alexandru Ianta

May 1, 2025

Abstract

Large-Language Models (LLMs) have shown promise in powering In-Context Learning (ICL) web agents designed to complete web tasks. However, state-of-the-art ICL agents continue to face challenges with tasks that require many steps to complete as well as element grounding (correctly identifying appropriate elements to interact with when applying an action). This work introduces a new methodology for creating LLM-enhanced web agents that make use of a semantically annotated navigational model constructed from user-interaction traces to mitigate these challenges. Through the use of the model, OdoBot outperforms WebVoyager, a state-of-the-art ICL agent, on a set of 50 real-world web tasks performed on the Canvas Learning Management System. OdoBot achieves 100% task success rate in comparison to 64% for WebVoyager while using the cheaper GPT-4o-mini model, in comparison with GPT-4o for WebVoyager.

Contents

1	Introduction	4
2	Related Work	6
2.1	ICL vs. Other methods	6
2.2	ICL methods	7
2.3	Element Grounding	8
2.4	Retrieval Augmented Generation	10
3	The OdoBot Methodology	11
3.1	Application Modeling	11
3.1.1	Trace Collection	11
3.1.2	Navigation Model Construction	12
3.1.3	Task Parameter Identification	15
3.1.4	LLM-based parameter annotation	18
3.2	Task Execution	19
3.2.1	Task Query Construction	19
3.2.2	Path Execution	21
4	Evaluation	22
4.1	Defining Evaluation Tasks	23
4.2	Generating Tasks	23
4.3	Defining Task Success	26
4.4	Results	26
5	Progress to Date & Timeline to Completion	28
6	Contributions	32
	References	34

1 Introduction

As Large Language Models (LLMs) have grown and become more capable, researchers are attempting to leverage them for the automation of web tasks. These tasks consist of a target website and a corresponding natural language description of a task to be performed on the website. The expectation is for a system, dubbed a ‘web agent’, to comprehend the task description and carry it out by interacting with the website. LLM-based web agents which rely on the LLM’s ability for reasoning within their *context window* are referred to in literature as In-Context Learning (ICL) methods. The context window is a set of tokens comprising the maximum input length for the underlying LLM [1–3].

Contemporary ICL agents face two key challenges. The first is *how to best use the limited context-window of the underlying LLM* to facilitate correct task execution. To facilitate this, ICL agents include some form of step history in the context window. This allows agents to ‘be aware’ of which steps were already taken, and reason about which action to take next [4–7]. However, this solution scales poorly as the number of steps required to complete a task grows, since the length of the history grows linearly with the number of steps taken. Thus the step history is an ever growing component of the information in the context-window, contributing to the degradation of ICL agent performance on tasks with many steps (long-horizon tasks) [4, 6]. The second problem is *identifying the correct elements of the target application to interact with* when applying actions. This problem is referred to as *element grounding* in literature. Existing methods for identifying target elements for action execution fall well short of oracles where humans interpret the LLMs’ element description and manually select the correct element [8].

This thesis provides a methodology for a model-centric approach for building domain specific web agents that aims to address the limited context window and element grounding problems. The approach is model centric in that *user-interaction traces* are collected for a target web application and used to construct a navigational model by mapping equivalent events from the traces onto vertices of the model. These vertices are connected by directed edges which reflect the order of events in the traces, to produce a graph of the interactions that are possible on the application. On this model, solving a task means finding a path that describes the interactions which fulfill the task objectives. LLMs are used to annotate vertices in the model that relate to entering information or making task-

specific decisions (like which option to select). These vertices are referred to as *task parameters*. While vertices representing API calls offer a pool of targets for paths (tasks) whose objectives invoke side-effects on the server side.

Together, (i) trace collection, (ii) model construction, (iii) parameter identification and (iv) parameter annotation constitute the *application modeling* phase of the method. Subsequently, *task execution* involves (i) using the natural language description of a task to identify relevant task parameters and target vertices, which are then used to find a path through the model. Next is path execution(ii), here the agent follows the interactions defined by the path, entering relevant information from the task description, utilizing an LLM to make task-specific decisions along the way.

These systems are implemented in the form of two components, the first is a browser extension dubbed ‘OdoX’ which facilitates trace collection, and the application of actions during task execution. Separately, a server called ‘OdoBot’ constructs the navigational model, processes state representations and sends back actions to OdoX during task execution.

In contrast to ICL methods, LLMs are used in a supporting role: to annotate the navigational model during its construction, and make decisions during single execution steps at runtime. This means that the limited context-window can be dedicated to providing information relevant to these simpler tasks, decoupling the size of the context from the number of steps in a task. Elements that are interacted with during a trace have their relative addresses in the Document Object Model(DOM) (their *xpaths*) recorded, this allows them to be precisely located during task execution, mitigating the element grounding problem. By analyzing the DOM snapshots captured during trace collection and identifying the vertices in the model with which those snapshots correspond, it is possible to create an index of *where* information can be found in the target application. The web agent can then navigate there to answer information-seeking tasks on the basis of the live information found at that location.

The following are the (completed and planned) contributions of this work:

1. A methodology and a software tool for collecting *user-interaction traces* from modern web applications and processing them into a *navigational model*.
2. A methodology for autonomously executing web tasks:
 - Capable of finding information by navigating the target application.

- Capable of inducing server-side side-effects to fulfill task objectives.
- Whose action space can be restricted by tainting vertices on the navigational model.
- Whose performance does not degrade at the same rate as ICL agents with the number of steps required to complete the task.
- Which leverages elements observed in traces and encoded in the navigational model to address the element grounding problem.

The rest of this document is laid out as follows. Section 2 provides an overview of ICL methods and how they make use of the LLMs limited context-window to drive web task execution. Section 3 provides definitions of web tasks, the navigational model, and user-interaction traces. It also describes how the navigational model is constructed from user-interaction traces and how task execution works leveraging and the model and LLMs. Section 5 provides an overview of the progress made to date, including systems for the automated execution of side-effect inducing tasks, and the Canvas Case Study. Section 5 gives a timeline for the higher-level work that remains to be done expanding the systems to support information seeking tasks.

2 Related Work

2.1 ICL vs. Other methods

ICL methods are distinct from agents whose underlying LLM has been pre-trained or fine-tuned for the execution of web tasks. This work is compared against ICL methods because they require less data and are competitive, or even outperform, methods using specially trained LLMs. Gur et al. showed that LLMs pre-trained on natural language corpora outperform those pre-trained on HTML understanding datasets [9]. Supervised Fine Tuned (SFT) methods which combine LLM fine-tuning with reinforcement learning techniques have been shown to be competitive with ICL methods on domain-specific tasks [8, 10]. However fine-tuning requires significant amounts of exemplars (in [10] models were fine tuned with 1,836 exemplars), and reinforcement learning approaches rely on well defined reward signals to evaluate the quality of different trajectories towards task completion. Such signals can be difficult to construct for real-world applications [10, 11]. In contrast, the full implementation of the proposed method requires only one demonstrative trace per type of task.

2.2 ICL methods

The typical ICL method setup is a web agent that is given a task description, and a representation of the environment (a state) after which it produces an action (clicks and keystrokes) that is applied to change the environment towards the completion of the task objective. [1, 2, 7]. One step of execution is a (representation, action) tuple.

Kim et al. provide an early example of such a system of prompts with their Recursively Criticizes and Improves (RCI) schema. RCI builds on the self-criticism capabilities of LLMs, in which they are prompted to identify errors with their own output, a process which is referred to as an RCI loop. The agent employs RCI loops to convert the input task into an plan, as well as produce executable actions. The agent maintains a list of previously executed actions to keep track of step history. More complex tasks with more steps result in an ever growing context inside the context-window. The authors cite ‘under-performance in tasks that require long-horizon planning’ as a failure point of the method [6].

Subsequent works in the ICL space have began leveraging external memory systems to facilitate knowledge transfer between similar tasks. Synapse [5] improves on the performance of other ICL methods through trajectory-as-exemplar prompting. When given a task for execution, Synapse first queries an external memory module to retrieve step histories (trajectories) of similar tasks. The top-k most similar task trajectories are then included in the context window to guide the agent’s planning. While Synapse includes the full trajectories of the related task in the agent’s context window, TRAD is an iteration upon the technique which only includes relevant steps from the related trajectories, reducing the size of trajectories in the context-window [12]. Interestingly, this only produced modest performance improvements over Synapse. Whether related steps or full trajectories are included, like RCI, the ever growing step-history continues to persist in the context-window.

Wilbur is a web agent that also makes use of a graph model but only during task execution. The model encodes the actions taken by the agent thus far, and is used to implement an error-backtracking mechanism. An LLM is used to analyze the state observation following an action and determine if the state corresponds with the reasonable execution of the planned action. If not, the execution graph is used to identify the URL of the most recent web navigation action, the agent reloads that URL and attempts to continue executing from there [13]. Wilbur’s execution plans are synthesized by

an LLM from previously seen successful and unsuccessful trajectories, whereas the proposed method computes execution plans by finding paths through a navigational model of the target application.

SteP is a web agent, that makes use of pre-defined policies ('find_commits', 'search_reviews', 'search_nearest_place') encoded in a policy graph (library) to guide task execution. SteP keeps track of applied policies in a stack, and uses a Markov Decision Process (MDP) to predict the next action. Actions can take the form of an action to be applied to the target application, or the invocation of a new policy from the policy graph [14]. However, while policies in SteP have to be manually defined, execution paths in the proposed methodology are derived from a model that is generated from user-interaction traces.

More recent works highlight the capabilities of web agents making use of Large Multi-modal Models (LMMs). These agents are capable of observing the state of the target application through screenshots. SeeAct, introduced by Zheng et al. is one such agent [8]. While they suffer from the same challenges with element grounding and limited context-windows, they do show promising performance uplifts over counterparts that use textual representations [4, 8].

WebVoyager is another LMM agent that is capable of interacting with both textual and screenshot representations of the target application's state. It leverages the ReAct prompting paradigm [15] to prompt the agent to 'produce a thought process' prior to generating the action [4]. He et al. constructed their own evaluation dataset comprised of 643 spanning 15 different websites for WebVoyager. With screenshot representations, WebVoyager achieved a task success rate of 59%. With textual representations, derived from the accessibility trees of webpages, WebVoyager achieves 40% task success rate. In their error analysis, He et al. identify longer trajectories resulting in lengthy contexts which hinder effective instruction following as contributing to failure cases for WebVoyager, echoing challenges with long-horizon tasks. Nevertheless, WebVoyager was chosen to benchmark the proposed methodology because it is part of the latest works in this area of research, requires no additional training, and has readily available and functioning code artifacts.

2.3 Element Grounding

Element grounding is the task of identifying the correct (target) element on the page to interact with when applying an action. Zheng et al. compare different state-of-the-art element grounding

approaches to a human oracle. They analyzed attribute-based, choice-based, and image annotation based element grounding comparing them against a human oracle.

Attribute-based grounding works by prompting the LLM to generate a detailed description of the target element’s attributes which are then used to find matching elements on the page. If multiple elements are found the LLM is prompted again to select one. With choice-based grounding, a separate model is trained to identify ‘promising’ candidate elements from the page. The model considers the task and the current step in task execution to identify candidates. An LLM then attempts to choose the correct target element from the candidates. In image annotation based grounding, bounding boxes are computed for candidates (produced by the same process as for choice-based grounding) in a screenshot, a separate model generates a label for each bounding box. The LLM is then expected to generate the label corresponding with the bounding box of the target element. Finally, the oracle baseline is established by having human annotators read the LLM’s description of the target element and manually select the element.

Zheng et al’s agent, SeeAct, was evaluated on a set of tasks, where the performance of the agent was measured by step success rate (SSR) (the number of successful steps taken towards task completion). The evaluation was redone with each grounding technique above. The oracle outperformed the next best method by around 20% achieving, 61% - 65% SSR. Choice-based grounding achieved 32% - 42% SSR, while image annotation and attribute grounding methods achieved 13% - 24% SSR and 12% - 19% SSR respectively. This demonstrates, not only that there is a significant gap between state-of-the-art grounding techniques and the human oracle, but also that ICL agent performance is correlated with the ability to solve the element grounding problem [8].

In the proposed methodology, the element grounding problem is solved by identifying the elements used during interactions with the application through the traces that are collected. All elements referenced in vertices along paths in the navigational model are derived from specific elements observed in the traces used to construct the model. Their xpaths are retrieved as necessary when applying actions during task execution. This also means that the agent will not interact with any element that does not appear in the navigational model, a characteristic that can be exploited to restrict the action space of the agent.

2.4 Retrieval Augmented Generation

Broadly speaking, web tasks can be split into two categories: information seeking tasks, in which the agent must navigate to find a specific bit of information, and side-effect inducing tasks, in which the agent exercises features of the target web application to create, update, or delete domain specific entities. For example, ‘Update the title of the “Geo Quiz 1” quiz in the Geology course to “Igneous Rocks Quiz”, would be a side effect inducing task since a domain object (the quiz) is being modified. ‘Find the due date of my latest chemistry assignment.’ would be an example of an information seeking task.

Retrieval Augmented Generation (RAG) systems solve information seeking tasks about some static corpus. Typically RAG methods are not designed to interact with web applications, and thus are not web agents or direct competitors to this work. However their prevalence and effectiveness in question-answering makes them worth mentioning when considering information-seeking tasks.

At a high-level RAG systems split apart a corpus into chunks and compute embeddings for each chunk during an indexing phase. At question-answering time the most relevant chunks are retrieved, and included in the prompt used to generate the answer to the question. The technique reduces hallucinations, and improves the quality of the answer by offering the LLM rich, relevant information related to the question from which it can generate its answer [16]. Other works extend the RAG technique by expanding the forms of data that can be retrieved for generation augmentation. GraphRAG [17] allows the retrieval of graph structures relevant to the task at hand. This kind of information can be especially useful when answering questions about highly relational data (for example, who reports to whom in an organization).

However such approaches depend on that computationally expensive indexing process, in which data is chunked and embedded. This makes RAG techniques a poor solution for executing information seeking tasks on database driven applications, where information is updated often [16, 18]. Each update would require re-indexing in order for the agent’s answers to reflect the new information. With many users creating and modifying data in real-time the approach becomes impractical.

The method proposed in this work introduces an agent based approach to information-seeking tasks. Instead of answering based on a cached version of data, the agent ‘looks up’ the location of similar data in the navigational model, and then navigates to that location to retrieve the latest

information on the subject.

3 The OdoBot Methodology

The methodology that this thesis puts forward consists of two phases, application modeling, and task execution. Application modeling consists of four sub-steps: trace collection, model construction, task parameter identification, and LLM-based parameter annotation. The output of this phase is a semantically annotated navigation model of the application used as the basis for task execution. Task execution has two sub-steps, task query construction, and path execution. During task query construction, the natural language description of the task is mapped to relevant vertices. In the task execution phase, these vertices are used to determine an executable path through the navigational model that fulfills the given task. The remainder of this section describes each step in detail.

3.1 Application Modeling

The objective of this phase is to construct and annotate a navigation model. The model is represented as a graph and tasks are path traversals through this graph.

3.1.1 Trace Collection

Traces can be collected either by observing and recording user interactions with the application (user-generated traces) or by recording the interactions of an integration-testing harness with the application (synthetic traces).

In our methodology, synthetic traces are collected using Selenium, a test automation framework [19] that can programmatically (i) start a web browser, (ii) configure and activate trace recording, and (iii) execute a series of scripted tasks, before finally (iv) terminating the trace recordings. This approach is leveraged to construct the navigational model for the case study described in section 4.

A *trace* $t = \{e_0, e_1, \dots, e_n\}$ is a chronologically ordered set of *events*. Events in a trace can be one of five types: click events, data entry events, API calls, location events, and DOM effects. Each event is associated with a unique ‘ID’ and a set of key-value pairs which are detailed in Table 1. $Text(s)$ refers to any text associated with the clicked element. $XPath(s)$ refers to the address of the

associated DOM element for click events and data entries. *Method* refers to the HTTP method of the API call. *Path* here refers to the API endpoint of the API call, where path parameters can be replaced with ‘*’ by consulting API documentation.

In addition to the information shown in Table 1, trace events also contain snapshots of the DOM at the the time the event was captured. These snapshots are the basis for the *trace vector database* which is analyzed to extract various kinds of information about the target application. To that end, the DOM snapshots are turned into embedded chunks, using a chunking technique described by Tan et al. [20]. This technique turns fine-grained DOM trees into course-grained chunks, embedded and stored in a vector database. The ID of the event from where each chunk originates is also saved. The event ID maps each chunk to a vertex in the navigational model.

Table 1: The types of user-interaction events that are collected through a browser extension to construct the application model.

Event	Triggered By	Properties	Example
Click Event	Clicking on buttons, list items, hyperlinks, etc.	xpath(s) text(s)	/html/body/div[3]/div[2]/div/div/div[1]/div Log In
Data Entry	Entering data into a text field.	xpath(s)	/html/body/div[3]/div[2]/div/span/div
API Call	Performing a non-GET HTTP request to the application backend.	method path	POST /courses/*/quizzes
Location	A URL change/page load/page transition.	path	/courses/*/assignments/*/edit
DOM Effect	A modification of the DOM by the application.	All events have a unique ID. Path parameters are replaced with ‘*’.	

3.1.2 Navigation Model Construction

The navigational model is a graph with two types of vertices and edges constructed from a collection of traces.

Event vertices represent collections of *equivalent* events observed in the traces. They are connected together by directed *event edges* (labeled ‘NEXT’) in a manner that respects the temporal order of the events observed in the traces. There are 10 sub-types of event vertices. The first five

sub-types correspond with the five different types of events that traces can contain (see Table 1). Checkbox vertices are detected via data entry events, when the data entry event was triggered by an ‘input’ element whose type is set to ‘checkbox’. Additionally there are ‘collapsed’ versions of clicks, data entries, checkbox, and dom effect vertices. These collapsed vertices appear as a result of parameter identification and are discussed in section 3.1.3. An event is equivalent to another event if its corresponding non-ID properties match. The only exception to this equivalence criteria are DOM Effects. Two DOM Effects are equivalent if the event preceding the DOM Effect and the event succeeding the DOM Effect in its source trace are identical.

Parameter vertices and *edges* are the second type of vertices and edges in the model. Parameter vertices appear in the model after task parameters are identified (section 3.1.3) and annotated (section 3.1.4). Parameter edges (labeled ‘PARAM’) connect the vertices of identified and annotated parameters to the event vertices in which they are detected. There are two sub-types of parameter vertices, *input parameters* and *object parameters*. Input parameters correspond with string input that is entered into the target application. Object parameters correspond with the selection of *domain specific objects* through the target application’s UI. A domain specific object is an instance of a named entity type defined in the UI of the target application. For example, a ‘course’ is a named entity type that appears in the Canvas UI, a particular course: ‘Psychology 101’, would be a domain specific object of that type, and thus a valid value for an object parameter associated with ‘course’ entities.

Figure 1 shows every type of vertex that can appear in a navigational model. Collapsed vertices arise from parameter identification described in section 3.1.3, while parameter vertices are discussed in section 3.1.4.

Model construction begins by iterating through the collected traces, and constructing the corresponding event vertices and edges, adding them one by one to the model until every trace has been processed. For every event, the model is queried for a vertex that is equivalent to the one that would be generated for the event. If no such vertex is found then a new vertex with the appropriate type and properties for the event is created and the event is associated with the vertex by adding the event’s ID to the vertex’s ‘instances’ list. If a vertex that is equivalent to the one that would be created is found, the event’s ID is simply added to that vertex’s ‘instances’ list.

A concrete example of this is shown in Figure 2 which contains two screenshots. The first is of the login screen of Canvas. Labels 1 & 2 highlight the data entries associated with entering the username and password respectively. Label 3 corresponds with the click event triggered by clicking the login button. Once the user has logged in, they encounter the second screenshot of the Dashboard. When the user finds themselves on the Dashboard screen, they may choose to access directly an existing course, or they may choose to see a listing of all available courses. Label 4a corresponds to the click event triggered by clicking the ‘Social Studies’ course card (former case). Label 4b corresponds to the click event triggered by clicking the ‘courses’ link from the sidebar (latter case).

The lower half of Figure 2 shows the navigational model constructed after the traces collected through the above interaction have been processed. A sample application model after the first phase of model construction. The model encodes the login procedure on the Canvas Learning Management System (LMS), followed by the transition to the dashboard and then either the selection of an existing course card, or a click on the ‘Courses’ link in the sidebar. The numbered vertices correspond with user interactions shown in Figure 2. Dark red vertices correspond with data entry events, teal vertices correspond with click events, green vertices correspond with API calls, dark blue vertices correspond with location events, and pink vertices correspond with DOM effects.

Figure 2 shows the model event nodes, constructed through a variety of traces of including login and dashboard interactions. In this model identical events from different traces are superimposed on one another because of their equivalence with each other, to construct the first six nodes. The divergence that occurs with interactions 4a and 4b reflects the differences between 2 or more traces used to construct the model. In one trace, the ‘Social Studies’ course card was clicked (4a), while in a different trace the courses link from the sidebar was clicked (4b).

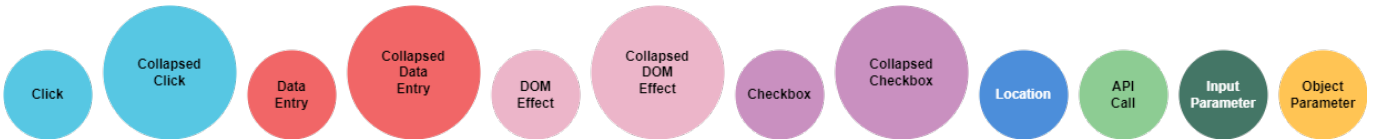


Figure 1: All the vertex types which can appear in the navigational model after model construction and LLM-based parameter annotation.

course has a quizzes section, other courses will as well. This is because the kinds of operations allowed on courses are the same for all courses.

2. When domain specific objects are presented to the user for selection, domain specific objects of the same type have the same internal DOM structure. For example, the course cards that appear on the dashboard have the same DOM sub-graphs to one another even if they have different styles and textual content.

The first assumption manifests in the form of parallel paths between two ‘anchor’ vertices in the model. Figure 3 illustrates three parallel paths in the model anchored by the dashboard location vertex on the right, and the ‘/courses’ location vertex on the left. These paths represent the selection of a specific course from those available on the dashboard. For the model to be useful, these paths have to be replaced with a single ‘generalized’ path describing the selection of any course that may appear on the dashboard, even ones that were not seen in the traces used to construct the model.

Two paths are considered parallel if they share the same starting vertex, ending vertex, length, and if the vertices at each index of the paths share the same event type. To achieve this, the model is scanned for vertices with out degrees greater than one. These vertices are potential *starting anchors* for parallel paths. A breadth-first traversal starts from the candidate starting anchor, building up a list of paths emanating from the vertex. Each path may not revisit a vertex already contained within the path. At each iteration of the traversal, the length of the paths in the list grows by one, and the list of paths expands whenever there are multiple outgoing edges from the current vertex to capture all possibilities. Then the path list is pruned such that the only paths that remain are the ones whose vertices’ event types match at each index with all other paths being explored. Finally, from the paths that remain, their last vertices are compared to each other to determine if they are not only of the same type, but in fact the exact same vertex. If that is the case, then an ending anchor has been found, and the search terminates. Otherwise, a new iteration of the search begins, extending the existing paths, adding new ones, pruning them, then checking for ending anchors once more. This process continues until the list of paths being explored is empty (all options are pruned).

If a set of parallel paths is found they are collapsed into a single path by collapsing all vertices at a particular index into a single *collapsed vertex*. For example, if three parallel paths of length four were found, the vertex at the second index of path one, the vertex at the second index of path two,

and the vertex at the second index of path three would all be merged into a new collapsed vertex. Then this process would repeat with the vertex at the third index of path one, etc. The vertex at the first index is the starting anchor and the vertex in the last index is the ending anchor. Thus in our example two collapsed vertices would be created. The properties of the vertices being collapsed are merged into the collapsed vertex's properties. For example, if each vertex was a click event with an xpath property, the collapsed click event would contain a list property 'xpaths' whose items would be the values of the xpath property for each vertex being collapsed. The search for parallel paths, and their subsequent collapses in the model continues until no parallel paths are found for any vertex in the model with an out degree greater than one.

When collapsing vertices that contain xpaths, the second assumption listed above allows the creation of a *dynamic xpath*. A dynamic xpath is a (prefix, wildcard, suffix) tuple. It corresponds with structures in the DOM which contain structurally identical elements. The prefix of a dynamic xpath is an xpath pointing to a container element (in Figures 3 and 4 this would be the element containing the dashboard course cards). The wildcard in a dynamic xpath identifies the type of tag that represents the root of a single item in the container (in Figures 3 and 4 this is a div tag). Finally, the suffix in a dynamic xpath describes a sub-structure that is common within each item in the container. Most often, this will be a link or button inside an item that was clicked (in Figures 3 and 4 this is the relative xpath of the anchor tag corresponding with the course name). Given a DOM, a dynamic xpath can be used to extract all instances of a particular kind of item from the page. For example, the dynamic xpath in Figure 4 can be used to extract all dashboard cards from a DOM.

While parallel paths correspond with the selection of some particular object in the target application, it is possible for any click event in the model to correspond with an object selection (dynamic xpath). Whether or not it does can be verified by checking if the parent(s) of the clicked element contains additional children with the same DOM structure as that of the clicked element. In the future, this *structural dynamic xpath extraction* technique will be applied after parallel path collapse as a part of parameter identification to ensure no object parameters are missed.

supplementary materials.

In this manner, all data entry vertices and collapsed click vertices are annotated with appropriate parameters. The result of parameter identification and annotation when applied to the sample application model shown in Figure 2 is shown in Figure 5.

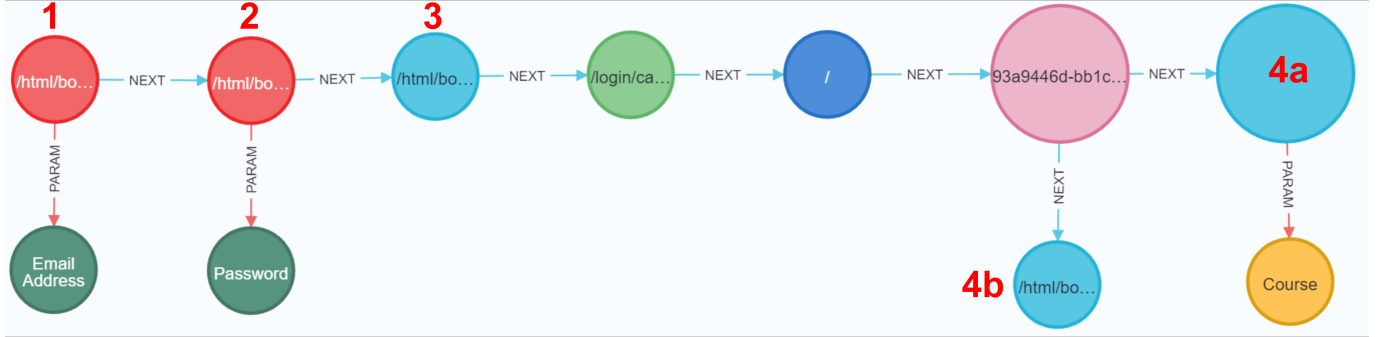


Figure 5: The application model, first shown in Figure 2, after parameter identification and parameter annotation. The data entries 1&2 have been annotated with input parameters ‘Email Address’ and ‘Password’ respectively, while the click event 4a has been annotated with the object parameter ‘Course’, reflecting that the click event corresponds with implicit course selection. Input parameters are dark green in color, while object parameters are yellow.

Once semantic labels are produced for every identified parameter in the navigational model, each label is embedded into a vector space and stored in the trace vector database. This database is queried during task query construction to match parameter values from the natural language task description to parameters annotated in the navigational model.

At this stage the navigational model is ready for task execution. An additional API is exposed to allow administrators to taint vertices and edges which should not be used during task execution.

3.2 Task Execution

3.2.1 Task Query Construction

The objective of task query construction is to produce a (path, parameters) tuple from the task description, which acts as input for the path execution mechanism described in section 3.2.2. During task query construction the task description is processed to identify which vertices must be visited to complete a particular task and to plot a path through the navigational model that visits all those vertices.

To determine the current ‘position’ of the agent in the model, a rolling window of recent interactions is stored, this is the *local-context*. When the agent is given a task, the local-context is processed as if it were a trace. The vertex corresponding with the last event in this trace is found in the model (via equivalence criteria) and becomes the starting vertex for task execution. If no interactions have taken place, the current URL of the agent is matched against location vertices in the model to identify the starting position.

To date, this is the only component of task query construction that has been implemented. What follows describes at a high-level the approach being investigated for the remainder of the task query construction process.

The objective is to identify a collection of target vertices in the navigational model that must be reached by a path from the starting vertex to complete the task. This could be done by developing natural language labels for possible target vertices, performing entity recognition on the task description to extract task parameters, and then matching the extracted parameters to target vertices. The matching could be decided by the cosine-similarity between vectorized embeddings of the target vertex labels and the extracted task parameters. This selects a subset of candidate vertices from all possible target vertices which align with the details of the task. For information seeking tasks the system can search the trace vector database created during the trace collection step to identify vertices where the sought for information might be found.

Next, a set of paths must be computed that begin from the previously identified starting vertex, and visit all candidate vertices produced by the matching. If more than one such path exists, a ranking function will be devised to identify the best one. This path, as well as the parameters extracted from the task description form the required input (path, parameters) tuple for the path execution mechanism.

Separately, an API will be exposed to allow the (un)tainting of vertices in the navigational model. Paths computed during task query construction will not be allowed to contain tainted nodes. This offers a degree of control over the behavior of the agent.

3.2.2 Path Execution

With the execution path and parameters determined in the previous step, the system is able to commence path execution. To begin, a cursor is initialized pointing to the first vertex in the path.

1. Advance the cursor to the next click, data entry, or checkbox vertex in the given path. The vertex pointed to by the cursor will be called the active vertex.
2. Determine if the active vertex has any edges connecting it to an input or object parameter vertex.
 - If the active vertex has an associated input parameter, then the active vertex is either a data entry or checkbox vertex. If it is a data entry vertex the associated input value and xpath of the input element into which the value should be entered is sent to the agent. If it is a checkbox, the agent is sent the desired checked-status of the checkbox and its xpath.
 - If the active vertex has an associated object parameter then executing this interaction will involve selecting an object from the target application UI. The dynamic xpath of the object parameter is retrieved and sent to the agent. The agent uses the dynamic xpath to capture a set of HTML snippets from the live DOM corresponding with the different object options. The options are presented as a multiple choice prompt to the LLM, along with a query string extracted from the task description. The LLM chooses the option that best matches the query string. The xpath of the chosen open is sent to the agent.
 - If the active vertex is not connected to any parameter it is a click vertex, and the agent is sent the xpath of the element to click.
3. The agent receives the instruction for an action and applies the action to the target application.
 - It enters the provided input value into the element at the specified xpath for a data entry instruction.
 - It finds the checkbox at the provided xpath and clicks it if necessary to match the provided checked-status.
 - It clicks on the element at the provided xpath for a click instruction.
4. In applying the action, the agent produces an interaction event, which it reports back to the server.

5. The server listens for interaction events matching the active vertex. When such an event is received it means the action has been applied to the application. If there are more vertices left in the execution path, return to step 1. Otherwise complete task execution.

For information-seeking tasks, an additional case will be added to step 2, if the active vertex is a target vertex for the information seeking task, a new kind of instruction will be sent to the agent, prompting it to scrape information from the live DOM that might be relevant to answering the question in the task description.

In addition to the automatic execution of tasks, a separate configuration for path execution will be developed in which the action to be performed is visually highlighted on the screen. This allows OdoBot to provide guidance to users seeking to learn how to perform a task on the target web application. Along side highlighting the relevant DOM elements, english language templates can be combined with vertex information to produce simple natural language descriptions of the action that needs to be taken by the user.

4 Evaluation

Currently, the system is capable of executing side-effect inducing tasks whose completion can be defined by reaching a single target vertex (API Call). As the task-query construction mechanisms are not yet implemented, tasks to be executed by the system must be specified in JSON format (Figure 6 shows an example OdoBot task definition). The experimental results from this case study therefore are an evaluation of the path execution mechanisms exclusively.

Canvas LMS was chosen as a target application to evaluate the proposed methodology on a realistic modern web-application. The objective is to define a set of side-effect inducing long-horizon tasks which are given to both WebVoyager and Odo Bot for execution.

WebVoyager is selected for comparison as it represents the current state of the art, requires no training phase, and has readily available code artifacts. While web agents like Wilbur (discussed in section 2) share more architectural similarities to the proposed methodology, in that they also require a ‘training’/setup phase to populate their demonstration banks and knowledge model, Wilbur fails to improve on WebVoyager’s performance when WebVoyager leverages it’s multi modal features to

observe the target application state via screenshots [13].

4.1 Defining Evaluation Tasks

Canvas is a learning management system, which is a kind of content management system, as such Canvas defines various different *types* of content that users can manage. A subset of these types are chosen: quizzes, assignments, pages, and modules, to produce a collection of side-effect inducing tasks in which create, update and delete operations are performed on each of the four selected content types. This results in twelve tasks that are shown in Table 2.

For web applications with API endpoints that conform to REST conventions the proposed definition of task success would be able to correctly differentiate create, update and delete operations for resources. Unfortunately, real-world applications like Canvas do not always conform to REST standards. As such, two tasks: deleting a quiz, and deleting a module are excluded from the evaluation task pool because they use the same API end points as editing a quiz, and editing a module, respectively (see Table 2).

Table 2: The collection Canvas tasks used for agent evaluation. Tasks 11 & 12 are excluded as they invoke the same API endpoints as tasks 5 & 7 respectively.

#	Task	Canvas API Endpoint
1	Create a new quiz.	[POST] /courses/*/quizzes/new
2	Create a new assignment.	[POST] /api/v1/courses/*/assignments
3	Create a new page.	[POST] /api/v1/courses/*/pages
4	Create a new module.	[POST] /courses/*/modules
5	Edit a quiz title.	[POST] /courses/*/quizzes/*
6	Edit a page title.	[PUT] /api/v1/courses/*/pages/*
7	Edit a module title.	[POST] /courses/*/modules/*
8	Edit an assignment title.	[PUT] /api/v1/courses/*/assignments/*
9	Delete an assignment.	[DELETE] /api/v1/courses/*/assignments/*
10	Delete a page.	[DELETE] /api/v1/courses/*/pages/*
11	Delete a quiz.	[POST] /courses/*/quizzes/*
12	Delete a module.	[POST] /courses/*/modules/*

4.2 Generating Tasks

Each of the tasks in Table 2 can be thought of templates for a particular kind of task. For example: ‘Login to Canvas with the username: example@gmail.com and password: test. Go to the psychology

course and edit the title of the quiz: Intro to Psychology to be: Psychology Quiz 1’, would be an instance of task 5 in Table 2. For each task, five unique instances are generated by identifying task parameters and populating their values with realistic data from publically available Canvas courses. In the example task instance given earlier, there are five task parameters: the username, the password, the course, the title of the quiz to be edited, and the desired title of the quiz.

The full list of the 50 task instances used to evaluate WebVoyager and OdoBot can be found in the supplementary materials.

Table 3: A list of courses downloaded from Canvas Commons, from which the model construction and evaluation tasks were generated. These courses were curated to ensure they contained the requisite number of quizzes, modules, pages, and assignments for their purpose. Two of each for model construction, and 5 of each for evaluation.

#	Course Data	Purpose
1	Environmental Science	Model Construction
2	English 9	Model Construction
3	Honours Biology	Evaluation
4	Psychology Semester 1	Evaluation
5	Spanish 101	Evaluation
6	US History	Evaluation
7	US Government	Evaluation

Since OdoBot performs task execution by following steps encoded in an application behavioral model, a collection of synthetic interaction traces is used to capture how each of the 12 tasks are performed on the application. These traces are processed to construct the application model as described in section 3. The traces used for model construction correspond to a collection of 20 task instances that is distinct from the 50 instances used for evaluation.

The synthetic traces leverage realistic course data extracted from Instructional Management Systems Common Cartridge (IMSCC) [21] files. These files contain all the data for a Canvas course, allowing the use of realistic values for the course objects (quizzes, pages, assignments, modules, etc.) that are created or modified during a synthetic trace. The IMSCC files are downloaded from publicly available courses published on Canvas Commons [22].

The result is a navigational model consisting of 130 vertices and 155 edges. Including six object parameters, eight input parameters, 35 click and eight collapsed click vertices, 46 DOM effect and 14 collapsed DOM effect vertices, 14 location vertices, seven data entry vertices, 13 API call vertices,

and one checkbox vertex.

To execute the 50 evaluation task instances, each instance is converted into a *task execution request*. OdoBot expects a task execution request to be a (id, target, userLocation, parameters) tuple. The id is a unique id for that particular execution, the target is the id of the target vertex (corresponding with the task API call) in the model, the userLocation is the URL of the currently active tab (eg: ‘where’ the task is being started from), the parameters are a list of input and object parameters required to complete the task. Each parameter is identified by the id of its corresponding parameter annotation vertex in the model, and its type (input or object). Input parameters listed in the execution request also include the parameter value, while object parameters include a query string used by the LLM to select the appropriate object. An example of a task execution request is shown in Figure 6.

Execution request tuples are created for all 50 evaluation task instances in the appropriate format for OdoBot and WebVoyager (see Figure 6). Once the evaluation task instances have been generated, they are inspected and an appropriate application state is manually induced in Canvas. That is, if there is a task to edit or delete a resource, that resource is created manually in Canvas. Once all necessary resources have been created, a backup is created of the Canvas database. Before an agent is evaluated, the Canvas database is restored from this backup, ensuring a ‘clean slate’. This restoration process happens only once, prior to the agent’s execution of the 50 evaluation tasks, it is *not* repeated between tasks during the experiment.

Each request is then executed and the success/failure of the task is recorded.

```
{
  "id": "4a6f8fd3",
  "target": "27669987",
  "userLocation": "http://localhost:8088/login/canvas",
  "parameters": [{
    "id": "f524ee66",
    "type": "InputParameter",
    "value": "example@ualberta.ca"
  }, {
    "id": "f450cae3",
    "type": "InputParameter",
    "value": "password"
  }, {
    "id": "9f19272c",
    "type": "ObjectParameter",
    "query": "World History"
  }]
}

{
  "web": "http://localhost:8088",
  "web_name": "Canvas LMS",
  "description": "Create a blank new quiz in a course.",
  "id": "1|WebVoyager|4a6f8fd3",
  "ques": "Use the username: example@ualberta.ca and
    password: password to login. Navigate to the
    World History course and create new quiz."
}
```

Figure 6: The same task in OdoBot format (left) and in WebVoyager format (right). The matching colors show how the same information is encoded in each format.

4.3 Defining Task Success

The two agents are quantitatively evaluated based on the number of tasks successfully completed. As WebVoyager supports both visual and textual state representations, both configurations will be included as reference points in the evaluation. Additionally, the experiment will be re-run using both the cheaper ‘GPT-4o-mini’ model, as well as the more expensive and capable ‘GPT-4o’ model.

For each evaluation task the url and HTTP method of the network request corresponding with successful task completion will be manually determined. During the experiment, network event traces will be captured and filtered for network requests matching the task-specific url and HTTP method. If such a network request is found, the task is marked as succeeded, otherwise the task is failed.

There is one exception to this criteria and that is for the ‘Delete an assignment.’ task. Upon clicking to ‘delete’ an assignment, Canvas displays a browser native confirmation modal, where the user is prompted to confirm or cancel the deletion of the assignment. This modal is not part of the DOM but rather of the browser’s own user interface. The ‘confirm’ button on this modal must be clicked before the deletion network request is fired. The current version of OdoBot does not instrument and capture these modals in the navigational model, similarly, WebVoyager is not equipped to interact with non-DOM elements of the application. For evaluation purposes this modal is sidestepped.

For OdoBot, a javascript function, overriding the default confirmation modal behavior is injected into the page to ‘auto-accept’ all such prompts. This is trivial to do since the OdoX extension already injects several scripts into the page. The same script injection is less straight forward to implement for WebVoyager. However, WebVoyager provides detailed logs and screenshots of executions, thus, for executions of the ‘Delete an assignment’ tasks, the logs and screenshots are manually inspected, and if WebVoyager clicks the ‘delete’ button for the appropriate assignment the corresponding task will be marked as succeeded even in the absence of the appropriate network request.

4.4 Results

Table 4 shows the aggregate success rate over all tasks for all agents. OdoBot achieves 100% success rate while utilizing a textual representation of the target application and the smaller and cheaper GPT-4o-mini model. WebVoyager struggles when using a textual state representation (the accessi-

bility tree) to observe the target application, completing just 2 of 50 tasks (4% success rate). While accessibility trees are generally meant to annotate intractable UI elements of a web application, the accessibility tree for Canvas does not seem to offer the required information to facilitate task execution. WebVoyager’s performance improves when using screenshots to interact with Canvas with the same GPT-4o-mini model that OdoBot uses for model construction and path execution, in this formulation, it completes 9 of the 50 tasks (18% success rate). Finally, a dramatic improvement in performance is achieved when WebVoyager uses screenshots to observe Canvas and leverages the more capable and more expensive GPT-4o model.

These results show that sets of side-effect inducing tasks which are unambiguous, definable *and* correctly defined on the navigational model can be effectively executed by the proposed system. In practice, the yet-to-be-implemented ‘Task Query Construction’ process, in which the natural language description of the task is converted into a navigational model based task definition (parameter and target vertices are selected and given values to produce execution requests as shown in Figure 6) will be prone to error. This error will propagate through, impacting the overall performance of the system. However, these results show that system performance need not be constrained by the execution mechanism.

Unfortunately, for LLM-based web agents, it is unclear how the ‘correct understanding’ of the task and the execution of the understood task can be separated into independent processes which can be evaluated separately. Therefore it was not possible to give WebVoyager the same advantage that OdoBot has in how clearly the task parameters and objectives are delineated.

Another interesting observation arises from inspecting the state of Canvas after an agent completes all 50 tasks. With OdoBot the application is in the exact state one would expect after performing the provided tasks following the sequences of operations gathered in the traces and encoded in the model. However, after WebVoyager completes the 50 tasks, one can observe a variety of unrelated actions that took place as the agent attempted to complete the task. For example, courses being transitioned into the published state. Inspecting the execution logs that WebVoyager produces revealed that the agent attempted to publish the course in an effort to find the assignment it was meant to edit. Listing 1 is extracted from the log of interactions with the LLM that resulted in this behavior.

1 `Thought: I have returned to the main course page for "US Government," but the`

```

    assignments are still not visible, and the course itself remains unpublished.
    Since there's an option to "Publish," doing so might make the course content,
    including assignments, visible. This may allow me to access and modify the
    necessary assignment. Action: Click [4786]

```

Listing 1: ‘Thoughts’ from WebVoyager execution logs, leading it to publish a course in an attempt to modify an assignment.

Since WebVoyager observes the application through screenshots where particular items may not currently be visible in the viewport, or accessibility trees that may be incomplete, it can mistakenly interpret this as a sign to engage in exploratory/unrelated behavior. This highlights the risk of performing unwanted actions inadvertently. Because OdoBot also leverages LLMs to select domain objects like assignments, quizzes, etc. at runtime, it is possible for it to choose incorrectly, especially in cases where the item specified in the task does not exist or is otherwise ambiguous. But it will not perform any operations that do not appear along the path to the target vertex for that task, bounding the ‘error space’ for unwanted behavior in contrast to ICL agents.

Table 4: Results of the Canvas task experiment showing the kind of LLM model used, the type of observation used on the target application, the total number of tasks and the success rate of each agent tested.

Agent	LLM Model	Observation Type	Tasks	Succeeded	Failed	Success Rate
WebVoyager	GPT-4o	Accessibility Tree	50	2	48	4%
WebVoyager	GPT-4o-mini	Screenshot	50	9	41	18%
WebVoyager	GPT-4o	Screenshot	50	32	18	64%
Odo Bot	GPT-4o-mini	HTML/DOM	50	50	0	100%

5 Progress to Date & Timeline to Completion

The following is a high-level overview of the progress made to date. Completed work is marked with a checkmark ✓.

1. Trace Collection

- Build a browser extension capable of capturing interactions with an arbitrary target application. ✓
- Capture Click, Data Entry, DOM Effect, Location, and API Call events. ✓

- Support the collection of scripted synthetic traces. ✓
- Compute embeddings of DOMSnapshots and store in a Trace Vector Database.
- Capture Data Entry in to complex UI widgets (ie: embedded text editors).

2. **Model Construction** ✓

3. **Task Parameter Identification**

- Identify Input Parameters via Data Entry vertices. ✓
- Identify Object Parameters via parallel paths pattern in navigational model. ✓
- Identify Object Parameters via structural dynamic xpath extraction.

4. **LLM-based Parameter Annotation**

- Annotate Input Parameters using input element HTML, HTML context, and sample input values. ✓
- Annotate object parameters using HTML snippets derived from dynamic xpaths. ✓
- Create parameter embeddings and store them in the Trace Vector Database.

5. **Task Query Construction**

- Identify the starting vertex via local-context. ✓
- Identify relevant target vertices from natural language task description.
- Perform entity extraction on task description to identify task parameter values.
- Match extracted parameter values with parameter vertices in the navigational model.
- Support information-seeking task path planning via Trace Vector Database.
- Allow administrators to taint/disable navigational model vertices/edges and support path planning that excludes tainted vertices.

6. **Path Execution**

- Use browser extension to interact with target application during task execution. ✓
- Support single target path execution. ✓
- Support multi-target path execution.
- Support information-seeking path execution.
- Support a ‘guidance’ mode in which users are visually guided along an execution path.

To date, the systems required to generate and collect traces, construct navigational models, identify and annotate task parameters, and perform automated execution for side-effect inducing tasks

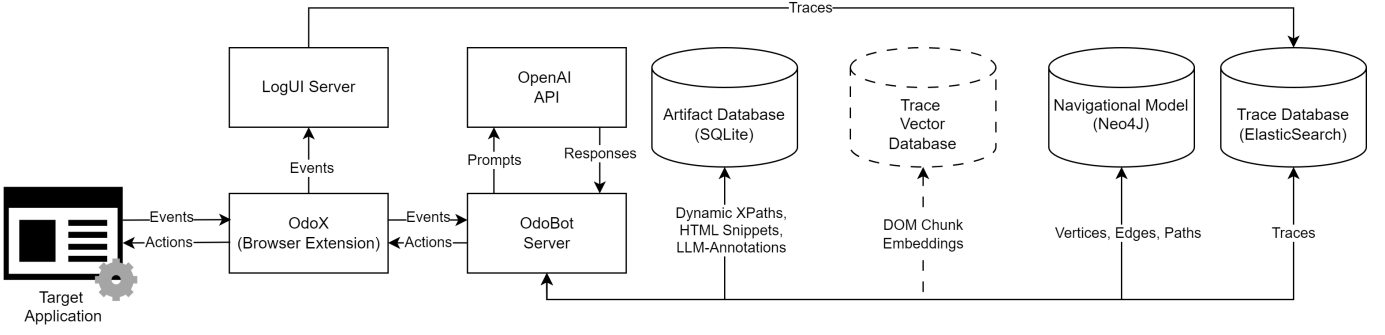


Figure 7: The high-level architecture of the developed systems. Components drawn with dotted lines have yet to be implemented.

have been implemented. Enough systems have been implemented to allow for the evaluation of the path execution mechanism on side-effect inducing tasks through a case study on the Canvas Learning Management System (LMS). In the remainder of this section, an overview of the implemented systems is provided before the case study is discussed in detail.

A browser extension dubbed OdoX was developed to facilitate the collection of interaction traces from any arbitrary website or web application. In addition, OdoX also acts as the client responsible for capturing and transmitting state representations and applying actions during task execution.

OdoX was built on-top of LogUI, a interaction logging framework developed by David Maxwell and Claudia Hauff [23]. The framework provides a client script that is injected into the page and then has the ability to transmit interaction logs to the LogUI server. This architecture assumes one has control over the source code and deployment of the application being observed because the LogUI client had to be embedded directly into the application. Significant effort was invested in integrating the LogUI client into the OdoX browser extension such that the extension injects the LogUI client into any arbitrary web page, thus removing the need to modify the target application.

While the LogUI server to which OdoX transmits traces to offers a basic storage solution, it did not offer a convenient way to extract or work with traces in bulk, so the appropriate infrastructure was developed to move traces into ElasticSearch. Kibana, an off-the-shelf web client for querying and analyzing data in ElasticSearch, was used to explore collected traces. Aside from collecting of traces, OdoX also acts as the client for task execution. Separate UI screens were added to the extension, which allow users to make task execution requests. During task execution the client works closely with the Odo Bot server via three websockets to transmit the application state to the server

and apply actions that are sent back from the server to the target application, thus facilitating task execution.

The server exposes APIs for model construction, processing task execution requests, creating synthetic traces, and identifying and annotating task parameters in the navigational model. For model construction, the server accesses relevant traces from Elasticsearch, and builds the graph model in Neo4J, a graph database [24]. The server performs parameter identification and annotation by interacting with the OpenAI API to execute the relevant prompts (detailed in section 3). The server uses a SQLite database [25] to store intermediate model construction artifacts such as dynamic xpaths, HTML snippets, and auxiliary information about object and input parameters. Through these means the server currently implements navigational model construction, including parameter identification and annotation.

With respect to task execution, the server currently implements the path execution mechanism with support for single target vertex executions. The server establishes three web sockets with OdoX to synchronize task execution processes. One socket is used for control (starting/stopping task execution), one socket is used to receive real time interaction events, and the last socket is used to transmit actions to be applied to the target application by OdoX.

What follows is the implementation and evaluation plan of the remaining systems in support of this thesis.

- **April 2025 - June 2025:** Implement missing task query construction features, this includes vertex annotations for API call vertices in the navigational model, and entity extraction for natural language task descriptions. Evaluation of the task query construction implementation will be facilitated by the evaluation tasks developed thus far. Publish results to relevant venues.
- **June 2025 - September 2025:** Implement missing systems for the execution of information-seeking tasks such as: computing embeddings for DOMSnapshots, storing them in the trace vector database, as well as expanding the path execution mechanism to extract task relevant data once the target vertex is reached. Evaluate the new systems using a set of evaluation tasks specifically oriented around information seeking. Publish results to relevant venues.
- **October 2025:** Implement tainting/disabled edge and vertex support, and construct a new set of side-effect and information seeking tasks to evaluate the tainting mechanism. Publish

results to relevant venues.

- **November 2025 - December 2025:** Develop natural language descriptions for each vertex along an execution path that can be displayed to users as part of the guidance mechanism that normally simply visually highlights the element that ought to be interacted with. Perform a user study in which OdoBot’s guidance capabilities are evaluated against conventional documentation for teaching users how to use an application.
- **January 2026:** Pending ethics approval, the evaluation of OdoBot as a guidance system for users will be performed, with results submitted to relevant venues.
- **February 2026 - March 2026:** Implement multi-target execution. Construct a new collection of tasks which require that multiple target vertices are reached during path execution. Re-evaluate task query-construction systems allowing for multi-target execution paths. Additionally, create a metric for task complexity, and develop an evaluation dataset of tasks that are compositional, requiring the completion of two tasks from the previous datasets, one after the other. Use this to demonstrate the scalability of the methodology to ultra-long task horizons. Publish results to relevant venues.
- **April 2026 - July 2026:** Study the generalizability of the method by constructing navigational models, and evaluation task datasets for Moodle, Wordpress, and other notable applications used to evaluate web agents. Publish results to relevant venues.

6 Contributions

Through the implementation of the systems described above, and their subsequent evaluation, this work offers an alternative architecture for web agents that grounds itself in observed real-world interactions with the target application through a navigational model. The methodology supports solving information seeking and side-effect inducing tasks. The use of a navigational model additionally makes it possible to exercise control over the manner in which tasks are executed by the agent through the use of tainted vertices and edges. This characteristic is promising for the development of ‘safety’ mechanisms that can prevent harm caused by inadvertently interacting with particular UI elements. To our knowledge, this level of control is novel in this area of research.

In summary this work contributes:

1. A methodology for collecting *user-interaction traces* from modern web applications and processing them into a *navigational model*. This model is represented as a graph of user-interaction events and information parameters provided by the user in the process of using the web application (Section 3.1).
2. A methodology for autonomously executing web tasks.
 - Capable of finding information by navigating the target application. The location of relevant information is identified through a similarity metric between the task and the embedded DOM snapshots saved during model construction (Section 3.1.1) with subsequent navigation driven by the path execution process described in section 3.2.2.
 - Capable of inducing server-side side-effects to fulfill task objectives. Appropriate API calls are determined through the task query construction process (Section 3.2.1, relevant parameters are identified apriori during application modeling (Sections 3.1.3 & 3.1.4), followed by path execution (Section 3.2.2).
 - Whose action space can be restricted by tainting vertices on the navigational model. The task query construction process omits tainted nodes when computing possible execution paths through the model(Section 3.2.1).
 - Whose performance does not degrade at the same rate as ICL agents with the number of steps required to complete the task. This mitigation is facilitated by driving task execution with a model/path based approach as opposed to keeping track of step history in the context-window of an LLM (Section 3.2.2).
 - Which leverages elements observed in traces and encoded in the navigational model to address the element grounding problem. This is a downstream benefit of driving execution through a model constructed from interaction traces (Section 3.1).

References

- [1] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, “Mind2web: Towards a generalist agent for the web,” in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 28 091–28 114. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/5950bf290a1570ea401bf98882128160-Paper-Datasets_and_Benchmarks.pdf
- [2] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin *et al.*, “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.
- [3] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou *et al.*, “The rise and potential of large language model based agents: A survey,” *arXiv preprint arXiv:2309.07864*, 2023.
- [4] H. He, W. Yao, K. Ma, W. Yu, Y. Dai, H. Zhang, Z. Lan, and D. Yu, “Webvoyager: Building an end-to-end web agent with large multimodal models,” 2024. [Online]. Available: <https://arxiv.org/abs/2401.13919>
- [5] L. Zheng, R. Wang, X. Wang, and B. An, “Synapse: Trajectory-as-exemplar prompting with memory for computer control,” 2024. [Online]. Available: <https://arxiv.org/abs/2306.07863>
- [6] G. Kim, P. Baldi, and S. McAleer, “Language models can solve computer tasks,” in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 39 648–39 677. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/7cc1005ec73cfbaac9fa21192b622507-Paper-Conference.pdf
- [7] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, “A survey on large language model based autonomous

- agents,” *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, Mar 2024. [Online]. Available: <https://doi.org/10.1007/s11704-024-40231-1>
- [8] B. Zheng, B. Gou, J. Kil, H. Sun, and Y. Su, “Gpt-4v(ision) is a generalist web agent, if grounded,” 2024. [Online]. Available: <https://arxiv.org/abs/2401.01614>
- [9] I. Gur, O. Nachum, Y. Miao, M. Safdari, A. Huang, A. Chowdhery, S. Narang, N. Fiedel, and A. Faust, “Understanding html with large language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2210.03945>
- [10] H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, Y. Li, X. Wang, M. Dehghani, S. Brahma, A. Webson, S. S. Gu, Z. Dai, M. Suzgun, X. Chen, A. Chowdhery, A. Castro-Ros, M. Pellat, K. Robinson, D. Valter, S. Narang, G. Mishra, A. Yu, V. Zhao, Y. Huang, A. Dai, H. Yu, S. Petrov, E. H. Chi, J. Dean, J. Devlin, A. Roberts, D. Zhou, Q. V. Le, and J. Wei, “Scaling instruction-finetuned language models,” 2022. [Online]. Available: <https://arxiv.org/abs/2210.11416>
- [11] H. He, W. Yao, K. Ma, W. Yu, H. Zhang, T. Fang, Z. Lan, and D. Yu, “Openwebvoyager: Building multimodal web agents via iterative real-world exploration, feedback and optimization,” 2024. [Online]. Available: <https://arxiv.org/abs/2410.19609>
- [12] R. Zhou, Y. Yang, M. Wen, Y. Wen, W. Wang, C. Xi, G. Xu, Y. Yu, and W. Zhang, “Trad: Enhancing llm agents with step-wise thought retrieval and aligned decision,” in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, ser. SIGIR ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 3–13. [Online]. Available: <https://doi.org/10.1145/3626772.3657788>
- [13] M. Lutz, A. Bohra, M. Saroyan, A. Harutyunyan, and G. Campagna, “Wilbur: Adaptive in-context learning for robust and accurate web agents,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.05902>
- [14] P. Sodhi, S. Branavan, Y. Artzi, and R. McDonald, “Step: Stacked LLM policies for web actions,” in *First Conference on Language Modeling*, 2024. [Online]. Available: <https://openreview.net/forum?id=5fg0VtRxgi>

- [15] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=WE_vluYUL-X
- [16] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” 2024. [Online]. Available: <https://arxiv.org/abs/2312.10997>
- [17] B. Peng, Y. Zhu, Y. Liu, X. Bo, H. Shi, C. Hong, Y. Zhang, and S. Tang, “Graph retrieval-augmented generation: A survey,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.08921>
- [18] M. Spiegel, “Chatbot-like natural language interface to a web-based information system,” 2024, Accessed: January 21, 2025. [Online]. Available: <https://is.muni.cz/th/j201g/>
- [19] Software Freedom Conservancy, “The selenium browser automation project,” 2025, Accessed: January 30, 2025 . [Online]. Available: <https://www.selenium.dev/documentation/>
- [20] J. Tan, Z. Dou, W. Wang, M. Wang, W. Chen, and J.-R. Wen, “Htmलगrag: Html is better than plain text for modeling retrieved knowledge in rag systems,” 2025. [Online]. Available: <https://arxiv.org/abs/2411.02959>
- [21] The NROC Project, “IMSCC,” Accessed: January 30, 2025 . [Online]. Available: <https://support.nroc.org/hc/en-us/articles/201025057-IMSCC>
- [22] Instructure, “What is canvas commons?” 2025, Accessed: January 30, 2025 . [Online]. Available: <https://community.canvaslms.com/t5/Canvas-Commons/What-is-Canvas-Commons/ta-p/1788>
- [23] D. Maxwell and C. Hauff, “LogUI: Contemporary Logging Infrastructure for Web-Based Experiments,” in *Advances in Information Retrieval (Proc. ECIR)*, 2021, pp. 525–530.
- [24] Neo4j, Inc., “What is a graph database,” 2025, Accessed: January 30, 2025 . [Online]. Available: <https://neo4j.com/docs/getting-started/graph-database/>

- [25] R. Hipp and Anonymous, “What is sqlite?” January 2025, Accessed: January 30, 2025 .
[Online]. Available: <https://www.sqlite.org/>